WebFGSR

Super Resolution and Frame Interpolation in WebGPU

Hangyu Zhang, Weike Qian, Yikai Li

CIS 5650 GPU Programming and Architecture, Fall 2026

Why It Matters

Problem with web rendering

- Native games upscale and generate frames with DLSS, FSR, XeSS, MetalFX
- Web pages can't call these SDKs; we found no web engine that generates frames
- Cost grows with resolution, and high-refresh displays need more frames

Our solution

- Port Mob-FGSR's SR + frame generation (FG) to WGSL in Babylon.js
- Fix ghosting where reprojection fails: disocclusions, thin structures

8.2 → ~30 ms

Project 4 Sponza, 5,000 lights, M4 Pro: 1080p vs. 4K

2.3 ms

Mob-FGSR interpolation + SR, 540p→1080p, Snapdragon 8 Gen 3, no NN at runtime

Implementation Goals + GPU Programming



Goals

- Temporal SR, 2x per axis
- Midpoint frame interpolation
- Reliability cues near occlusions, thin edges
- Optional: re-shade unreliable regions

GPU programming

- Hand-written WGSL compute passes
- Atomic motion splatting (32-bit atomics)
- Shared-memory tiles, f16, subgroups
- Per-pass timestamps on one queue

Timeline



Milestone 1
Nov 11

Super Resolution
Babylon inputs, reference capture, temporal SR

Milestone 2
Nov 18

Frame Generation
Motion splatting, hole filling, SR + FG end to end

Milestone 3
Nov 30

Quality + Speed
Reliability cues, ablations, optimization

Final
Dec 7

Demo
Live demo, videos, results

Evaluation vs. native full res: PSNR, SSIM, FLIP, cue ablations, per-pass and total frame time

References / APIs + Third-Party Code

APIs + platforms

- WebGPU + WGSL
- Babylon.js ComputeShader
- TypeScript, Vite
- Desktop Chrome

Third-party code

Babylon.js: scene, depth, velocity

Mob-FGSR: adapted with approval

NVIDIA FLIP: metrics

References

Mob-FGSR, SIGGRAPH 2024

Edge-aware supersampling, TOG 2025

Predictive sparse shading, SIGGRAPH Asia 2025